

Using Generative Artificial Intelligence to support Undergraduate Learning in Computing in Chemical Engineering Course

Dr. Sihe Zhang, Kettering University

Sihe Zhang, Assistant Professor, Dept of Chemical Engineering, Kettering University, 2024-present West Virginia University Institute of Technology, 2020-2024 Postdoctoral Fellow, Georgia Institute of Technology, 2018-2020

Torsten Maier, Kettering University

Dr. Torsten Maier is an assistant professor in the Department of Industrial & Manufacturing Engineering. His research areas include engineering education, AI, and esports science. His teaching focuses on human factors engineering, team and project-based courses, and engineering economics.

Dr. Susan Farhat, Kettering University

Using Generative Artificial Intelligence to Support Undergraduate Learning in Computing in Chemical Engineering Course

Abstract

Computing skills are essential for modern chemical engineers, while many undergraduate students struggle with converting mathematical models into working algorithms, as well as executable code. Generative artificial intelligence (GenAI) presents both significant challenges and transformative opportunities for higher education in Chemical Engineering and other engineering disciplines. Within this evolving landscape, the present work discusses efforts to integrate GenAI as a learning facilitator, rather than a solution provider, to support students' learning in a sophomore-level Chemical Engineering Computing course—a course centered on essential computational and simulation tools such as Excel with embedded VBA, MATLAB, and other discipline-specific software. A series of learning activities was developed on the topics of VBA programming and coding with MATLAB to solve Chemical Engineering problems, with deliberately designed GenAI attempts to enhance conceptual reinforcement, learning confidence, active learning, and reflective critical thinking. These activities were anchored with GenAI to provide instant feedback on code, generate robust calculation templates, assist with algorithm development, etc. These activities were paired with structured reflection prompts that guide students to critically examine the role of GenAI, recognize its limitations, propose how to make improvements, and develop responsible usage habits. The observation of students' performance in this study suggests that, within the scope of the studied teaching/learning activities, infusing GenAI into instructional/learning processes can improve students' responsiveness, reduce computational anxiety, increase comprehension of syntax, and afford personalized-iterative learning. This work investigates the GenAI application as an instructional aid, offering case studies and experiences that can facilitate broader adoption of pedagogical practices enhanced by AI in Chemical Engineering education. Future work includes extending the semesters to encompass more sophomore students, expanding the types of computing problems, refining the assessment approaches, and potentially integrating GenAI in more Chemical Engineering courses and other engineering computing courses.

Introduction

Computational proficiency has always been serving as the foundational skill in chemical engineering practice, including data collection and analysis, modeling, simulation, and process optimization. Chemical engineering undergraduate programs typically introduce computing through courses that incorporate tools such as MATLAB, Excel with VBA, and AspenPlus. While understanding their importance, freshman/sophomore undergraduate students often persist in these courses/topics as challenging and intimidating, particularly for those with limited prior programming experience [1], [2].

Chemical Engineering undergraduate students are frequently challenged in their computing courses or problem-solving computing procedures, such as translating the problem statement texts and physical models into algorithms, building connections and transitions between formulas and actual solutions, debugging syntax and logic errors, not to mention coping with the fast pace of course content and managing frustration when support is not always on-the-spot. Traditional instructional approaches for computing courses, such as lectures, static examples, time-consuming debugging, professors' manual code grading, often lead to delayed feedback on students' coding work, especially those not-so-sufficiently-provisioned students. Frequently, freshman/sophomore students struggle during the learning cycles, particularly with the error-propagating nature of computing, where small mistakes or confusion can lead to cascaded difficulties in the following learning stages and problem-solving [1], [3].

As Generative artificial intelligence (GenAI) has developed dramatically in the past several years, a lot of pedagogical studies and practices in engineering education have explored many directions using AI, such as AI-assisted tutoring, automated feedback, and intelligent learning systems, such as the Chatbots in the Material and Energy Balance Course at Columbia University [4]. GenAI opens a new avenue of pedagogical tools distinguished by natural language interaction, on-demand explanation, customized examples, rapid feedback, and code generation and revision. There has been vigorous debate on the related ethical concerns, such as critical thinking undermining and plagiarism; however, the advent of the GenAI era is undeniable. As such, explicit and well-rounded GenAI usage policies that emphasize academic integrity should be set forward in order to maintain the quality of higher education. Today's students will use GenAI more extensively and profoundly than today's educators and, therefore, must be intentionally and thoroughly trained to use it ethically and responsibly. Additionally, the literature suggests that structured and reflective use of GenAI can promote deeper understanding by encouraging students to actively engage with content, cultivating self-regulated learning through faster feedback and reflection on their own learning processes [5], [6].

This work explores the intentional incorporation of GenAI into a sophomore-level Chemical Engineering Computing course. Rather than allowing unrestricted AI use or NO-AI use, the course design emphasized guided, reflective, and transparent interactions with GenAI to support learning objectives related to algorithm development, problem-solving, and computational thinking, while fostering students' awareness of the limitations of GenAI application and the extent to which they would adopt AI in their work.

Methods

The study was conducted in a required 200-level Chemical Engineering undergraduate computing course at Kettering University, typically taken in the sophomore year. This course introduces: computing techniques, basics of computer programming and its applications to the solution of chemical engineering problems, advanced spreadsheet (Excel) applications and VBA, logic flowchart and MATLAB programming fundamentals, basic toolset to tackle common tasks like numerical integration, curve fitting, regression analysis, ODE solutions, and data visualization and interpretation, in addition, industrial process simulation software such as AspenPlus.

By the end of the course, students should be able to develop and implement algorithms to solve chemical engineering problems, using data to construct predictive mathematical models and evaluate and modify the models, interpret computational results in a chemical engineering context, understand basic computer programming concepts such as variables, assignments, looping and conditional executions, and (in AspenPlus) to create simulation of unit operations and a chemical engineering process. Students should also be able to reflect critically on computational tools and their limitations.

The integration of GenAI as a teaching assistant was explicitly aligned with these objectives. Not all class content and topics were infused with GenAI assistance, e.g., basic Excel formulating, regression toolkit, solver, PolyMath, and AspenPlus simulation of process were not included in this study. The involved class activities were guided by the following principles:

1. Baseline: GenAI is not a solver, but a facilitator;
2. Reflection: The student must evaluate GenAI-generated results;
3. Transparency: The student must disclose the full details of GenAI use;
4. Validity: The Student is responsible for the correctness of the solution.

These policies were discussed during class sessions when GenAI was prompted to be used.

This work was determined to be exempt from further review by the Kettering University IRB.

The Informed Consent Form was distributed before data were generated and collected, and signed by students who were willing to participate.

Example Learning Activities

First, students were introduced to VBA macros in Excel, user-defined functions, User-Forms, and MATLAB coding for four weeks, then they were challenged with assignments of: 1. coding a User-Form (a medical diagnosis User-Form for fever and medicine based on the patient's age and temperature) in VBA, and 2. coding a MATLAB program to find the solution of an ODE system on a chemical reactor design problem.

Activity 1: Algorithm Development with GenAI Prompting

Students were asked to outline the flowchart for the medical diagnosis form. GenAI could be used to review the algorithm logic and suggest redundant or missing steps. Students then discussed their algorithms in class and assignments to explain what changes were made and why.

Assignment statement:

Create a flow chart for the medical diagnosis User-Form, and ask AI to review the algorithm logic, and suggest redundant or missing steps. Use your own words to explain what changes were made and why.

Problem statement [7]:

Create a medical diagnosis User-form in Excel-VBA that makes decisions that if a patient: 1. Should take medicine, 2. Can take Aspirin (when age is older than 12) and 3. Need Emergent medical help (when temperature is above 41.1 °C), based on the input values of the patient's temperature and age, with the following criteria:

I. For babies less than 1 year old, a temperature over 38.3 °C should receive medication;

II. For children less than 12 years old who are not babies, a temperature over 38.9 °C should receive medication;

II. For people over 12 years old, a temperature over 38.3 °C should receive medication.

Disclaimer: There is no universally accepted criterion for deciding whether someone's body temperature is high enough to require medication. The values given here are sometimes used, but certainly are no replacement for qualified medical advice.

Activity 2: Code generation, Debugging and Feedback

Students were asked to code by themselves first (VBA and MATLAB). Then, GenAI was permitted to: generate the codes using VBA or MATLAB, then identify syntax or logic errors in their own codes, and explain error messages.

Assignment statement:

1. Based on your corrected flow chart, create and code the medical diagnosis User-Form by yourself first in VBA. Then ask AI to generate the codes using VBA. Compare your procedure/codes with the AI-generated, then identify which should be adopted and modify your form accordingly. Explain the difference.

2. Develop a MATLAB program to solve the following Ordinary Differential Equation system, which describes a reactor. Then use Copilot that's built into MATLAB for help, to create a MATLAB script (.m file) to solve this ODE equation system. Compare the codes.

Problem statement for the 2nd assignment:

A non-isothermal batch reactor is described by the following two ordinary differential equations:

$$\frac{dC}{dt} = -e^{\left(-\frac{10}{T+273}\right)c}$$
$$\frac{dT}{dt} = 1000e^{\left(-\frac{10}{T+273}\right)c} - 10(T - 20)$$

t is the reaction time, C is the concentration of the limiting reactant in the reaction, and T is the reaction temperature. At $t = 0$, $T = 15$ degrees Celsius, and $C = 1$ mol/L, determine the concentration and temperature profiles with the change in the reaction time in this reactor.

Activity 3: Reflection on GenAI utilization experience and its limitations

Students were asked to document their whole progress of using GenAI including their original prompts, the GenAI outcomes, their modified prompts for finer solutions, as well as discuss their experience and share insight on proposing how to responsibly using GenAI in Engineering calculation, such as what knowledge was required to detect the inaccuracy in GenAI answer, what GenAI helps them most to solve the problem, and how they should approach coding and computing problems using GenAI in the future.

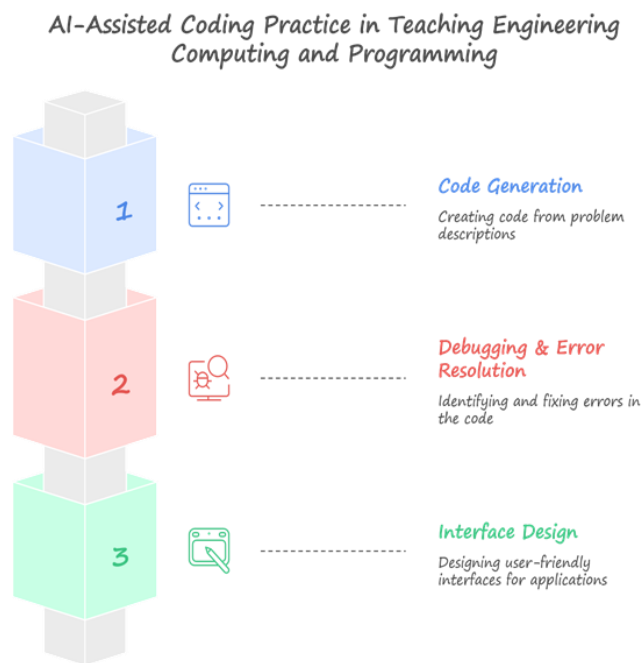


Figure 1. GenAI-Assisted Coding Practice in Teaching Engineering Computing

Assessment:

Students' learning and perceptions were assessed using: I. reflective essays including the logic, transparency, and thoughts conveyed in their narrative, though technical writing skills were not the focus; II. effectiveness of their coded programs and the correctness of their results, and III. assignment performance, for instance, the correctness of their flowchart logic, redundant variables or loops, and missing steps.

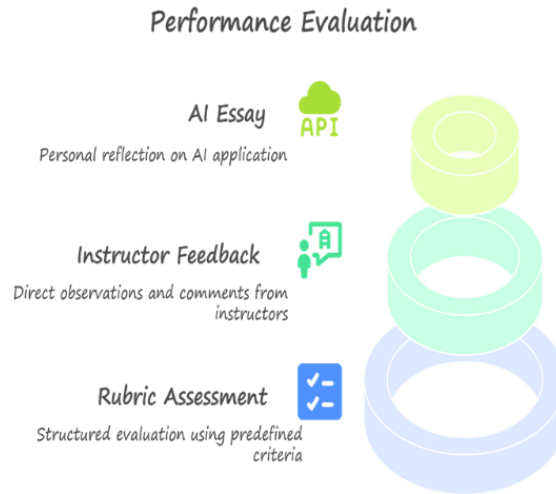


Figure 2. GenAI-Assisted Coding Practice Effectiveness Evaluation in Teaching Engineering Computing

Results

Based on predefined rubrics, students using GenAI assistance demonstrated a 0–50% increase in task completion rates, compared to students' performance in previous terms, for beginner-level coding challenges. The wide range was due to the variety of students' previous experience of coding, and personal learning habits such as reflection and revision.

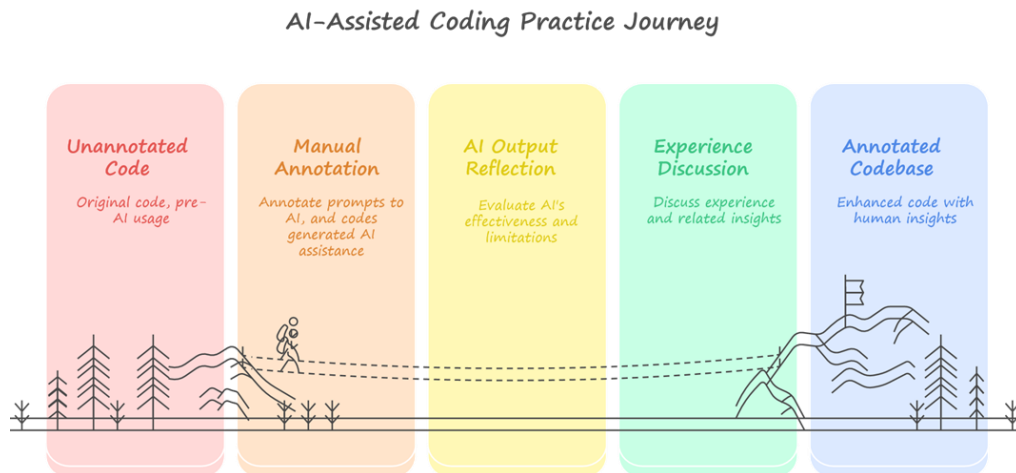


Figure 3. GenAI-Assisted Coding Practice Journey in Teaching Engineering Computing

Compared to the students' performance in class reaction and assignment score in previous NO-AI use terms, code quality improved significantly with respect to logic flow, code structure, and syntax formatting, though conceptual understanding (such as conditional execution priority, iterations, and counter variables setup) required learning without AI in advance.

Students' feedback on what GenAI helps include:

--- "AI helps when you want a quick explanation on an error you are having."

--- "It (ChatGPT) reassured me that my code was correct, as I felt like I was going out on a whim."

---"What GenAI helps me the most with is double-checking my work and reassuring myself that I am doing the right things and that I am on track to finish the problem correctly."

---"It (CoPilot) was very fast and produced a solution that matches the solution from my PolyMath program. It was also able to tell me about the code it wrote when I asked about it."

Students' feedback on what GenAI doesn't help include:

---"AI is not helpful when creating the user form visuals, such as the text boxes and the labels."

---"AI didn't follow the flowchart exactly."

---"With AI being extremely helpful at rapid speed, it is extremely vulnerable to mistakes. Therefore, some general knowledge is required before using the AI on this topic."

---"One problem I think that Copilot AI has is when I asked it to find a way to make the code better, it didn't try to change the code but rather add more features to the code that was already there."

Example of a student's work

Code made by Me:

Standards:

MedResp = "No Aspirin"

ActionResp = "No Meds"

EmergResp = ""

Emergency:

If T>41.1 Then EmergResp = "Get Help!"

If Age >21 Then MedResp = "Aspirin OK"

If Age<1 and T>38.9 Then MedResp = "Give Meds"

Example of students' work with GenAI debugging

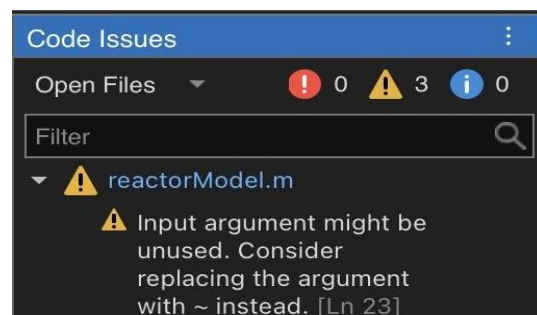
The **Runtime Error 424: Object required** in VBA occurs when you try to reference an object that doesn't exist or hasn't been properly set. In your case, the error happens on the line:

txtMeds.Text = M

where M is defined as:

Dim M As String
M = "No Meds"

This indicates the variable M is fine, but **VBA cannot find the object txtMeds**. This is not an issue with M; it's an issue with how txtMeds is referenced or initialized.



Students showed improved ability in explaining their codes during class discussion, correcting missed steps, and interpreting the logic of syntaxes and the solution results, indicating gains in conceptual understanding. More active engagement in class was observed, showing that these activities helped with building students' computing confidence. Essays also showed students felt less unsettled when initiating coding a program for a problem, confirming the code correctness, and GenAI reduced the time they spent stuck on the debugging process and syntax errors. Some students noted increased awareness of the limitations and potential deficiencies of GenAI-generated solutions.

Examples of Students' shared experience and insights on using GenAI:

---"I don't use the exact work from its (ChatGPT) responses, but rather mostly use it to figure out what I was doing wrong."

---"If you have no idea on what the topic is about, it will not be very helpful as any mistakes it makes will go unnoticed."

---"Understanding what you're working on is important when working with GenAI, as when issues arise you won't actually understand them if you don't understand the produced responses."

---"It's most definitely a useful tool that can do away with tedious work or double checking what you have, but knowing what you're doing will always be of utmost importance."

---"I've used different GenAI for various things before, but being able to use it at this level for a class greatly increased my perspective on how much the prompt you give GenAI matters."

---"It isn't at all immediately obvious why this made it work (despite the error) over the previous prompt, demonstrating the importance of understanding what exactly it is you're working with. Without an understanding of why the code works (or doesn't), you have no control over what is produced or what you're doing."

---"An insight is that GenAI does not inherently mean that the answer is right, the responsibility lies in you to check the answer and validate it yourself. A responsibility is to use GenAI as a tool, not a substitute. It is a good reference, like a textbook, but not a replacement for knowledge."

Certain challenges were observed during the study, including: the limited variety of problems used from Chemical Engineering computing, the limited time span/course sessions involved, the limited number of total students engaged, the variability in quality of GenAI responses, instructor time needed to design assignments and the design of grading rubrics to ensure sufficient scaffolding that can prevent students' over-reliance on GenAI.

Despite these limitations, the implementation provides valuable insight for integrating GenAI into chemical engineering computing instruction. Based on the implementation and outcomes of this study, integrating GenAI in Chemical Engineering computing course to facilitate students learning are suggested to: explicitly declare GenAI usage rules in both syllabus, class time and assignments, include rationale for requirements of assignments, embed reflective section and require

documentation of the GenAI usage in each practice, encompass learning to use GenAI with verification step and treating GenAI literacy as a clear learning outcome. Together, these practices can help an instructor utilize GenAI as a learning support tool while minimizing the students' over-reliance and also maintaining the development of their core computational and problem-solving skills.

Conclusion and Future Work

This study shows that infusing GenAI with carefully designed class activities and clear GenAI use policies into undergraduate Chemical Engineering Computing courses can support student learning experience with enhanced responsiveness, coding confidence, and GenAI awareness. This work highlights harnessing GenAI as an instructional aid and emphasizes accountability. Engineering educators can leverage the strengths of GenAI while mitigating potential risks. Future work will explore learning outcomes across multiple cohorts, longitudinal impacts on student performance, refining activity design with a variety of problem types, and the development of consistent assessment frameworks for more AI-assisted learning in Chemical Engineering courses.

Reference

- [1] M. Arjmandi *et al.*, Embedding computer programming into a chemical engineering course: The impact on experiential learning, *Education for Chemical Engineers*, vol. 43, pp. 50–57, 2023
- [2] F. A. Pirzado *et al.*, Assessing Computational Thinking in Engineering and Computer Science Students: A Multi-Method Approach, *Education Sciences*, 15(3), 344.
- [3] S. Yang *et al.*, Decoding Debugging Instruction: A Systematic Literature Review of Debugging Interventions, *ACM Transactions on Computing Education*, vol. 25, no.4, 45, Nov 2024
- [4] C. V.H.-H. Chen, and S. Ratanaalert, Development of an MEB Novice Chatbot to Improve Chemical Engineering, in *2025 ASEE Annual Conference & Exposition Proceedings: Leveraging AI and Computational Tools for Enhanced Learning*, Montreal, QC, June 22-25, 2025
- [5] E Kasneci *et al.*, ChatGPT for good? On opportunities and challenges of large language models for education, *Learning and Individual Differences*, vol. 103, April 2023, 102274
- [6] X. Xu *et al.*, Enhancing self-regulated learning and learning experience in generative AI environments: The critical role of metacognitive support, *British Journal of Educational Technology*, vol 56 (5), pp. 1842-1863, Sep 2025
- [7] R. W. Larsen, *Engineering with Excel*, Pearson, 2018, 5th Ed., ISBN – 9780134589664 (Paper), ISBN - 9780137518708 (Digital), Chapter 14, pp. 565-603.